

CSI2110 – Review Notes

Arrays

Definition 1 (Array Memory Layout)

In memory, array elements are stored contiguously. For an `int[]` in Java, each element is typically 4 bytes apart.

```
1 int[] arr = {10, 20, 30, 40};
```

Array Example in Memory

Address	Value
1000	<code>arr[0] = 10</code>
1004	<code>arr[1] = 20</code>
1008	<code>arr[2] = 30</code>
1012	<code>arr[3] = 40</code>

Singly Linked Lists (SLList)

Definition 2 (Singly Linked List)

Each node stores a value and a reference to the next node. Nodes are not required to be contiguous in memory.

Singly Linked List Node Layout

Address	Value
1000	<code>[10 next → 5000]</code>
5000	<code>[20 next → 2400]</code>
2400	<code>[30 next → null]</code>

Operation: Insert at Position *i*

Insert a new node so that:

- The new node points to the old successor node.
- The previous node now points to the new node.

```
1 void insertAt(int i, int value) {
2     Node prev = head;
3     for (int k = 0; k < i - 1; k++) {
4         prev = prev.next;
5     }
```

```

6     Node n = new Node(value);
7     n.next = prev.next;
8     prev.next = n;
9 }

```

Big Question 1 (Tail Pointer in SLList)

Q: Suppose we want to quickly insert or delete the last node. Is it sufficient to store only a tail pointer?

A: A tail pointer helps with inserting at the end in $O(1)$ when we already know the tail. However, deleting the last node still requires locating the second-last node, so in a singly linked list deletion at the end is not $O(1)$ unless extra structure is maintained.

Doubly Linked Lists (DLList)

Definition 3 (Doubly Linked List)

A doubly linked list stores **previous** and **next** references in each node, allowing two-way traversal.

Doubly Linked List Node Layout

Address	Value
1000	[null ← previous 10 next → 5000]
5000	[1000 ← previous 20 next → 2400]
2400	[5000 ← previous 30 next → null]

Operation: Delete Node at Position i

Deleting node i means:

- Find the target node.
- Redirect the previous node's **next** pointer.
- Redirect the next node's **previous** pointer.

```

1 void deleteAt(Node x) {
2     if (x.prev != null) {
3         x.prev.next = x.next;
4     }
5     if (x.next != null) {
6         x.next.prev = x.prev;
7     }
8 }

```

Important: In DLList, delete is cleaner because each node already has direct access to both neighbors.